Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92410133020**

Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:
Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:
Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:
Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly
actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word	stemmed words
0	connect	connect
'I	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92410133020**

Stopword Removal

Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", ‘‘the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?", you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Program (Code):

To be attached with

1. Tokenization
2. Filteration
3. StopWords Removal
4. PoS Tagging
5. Noun Phrase Chunking
6. Dependancy Parsing.

Code:

```
!pip install nltk
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker')
nltk.download('words')
nltk.download('punkt_tab')
nltk.download('maxent_ne_chunker_tab')

from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk import pos_tag, ne_chunk

text = """Natural Language Processing is an important field of Artificial Intelligence.
It helps machines understand human language."""

sentences = sent_tokenize(text)
words = word_tokenize(text)
```

```
print("Sentences:", sentences)
print("Words:", words)

words_filtered = [word for word in words if word.isalnum()]
print("\nFiltered Words:", words_filtered)

stop_words = set(stopwords.words('english'))
words_no_stop = [w for w in words_filtered if w.lower() not in stop_words]
print("\nAfter Stopword Removal:", words_no_stop)

ps = PorterStemmer()
stemmed_words = [ps.stem(w) for w in words_no_stop]
print("\nStemmed Words:", stemmed_words)

pos_tags = pos_tag(words_no_stop)
print("\nPOS Tags:", pos_tags)

chunk = ne_chunk(pos_tags)
print("\nChunking:\n", chunk)

!pip install spacy
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp(text)
print("\nDependency Parsing:")
for token in doc:
    print(token.text, "->", token.dep_, "->", token.head.text)
```

Results:

To be attached with

```
Sentences: ['Natural Language Processing is an important field of Artificial Intelligence.', 'It helps machines understand human language.']
Words: ['Natural', 'Language', 'Processing', 'is', 'an', 'important', 'field', 'of', 'Artificial', 'Intelligence', '.', 'It', 'helps', 'machines', 'understand', 'human', 'language', '.']

Filtered Words: ['Natural', 'Language', 'Processing', 'is', 'an', 'important', 'field', 'of', 'Artificial', 'Intelligence', 'It', 'helps', 'machines', 'understand', 'human', 'language']

After Stopword Removal: ['Natural', 'Language', 'Processing', 'important', 'field', 'Artificial', 'Intelligence', 'helps', 'machines', 'understand', 'human', 'language']

Stemmed Words: ['natur', 'languag', 'process', 'import', 'field', 'artifici', 'intellig', 'help', 'machin', 'understand', 'human', 'languag']

POS Tags: [('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('important', 'JJ'), ('field', 'NN'), ('Artificial', 'NNP'), ('Intelligence', 'NNP'), ('helps', 'VBZ'), ('machines', 'NNS'), ('understand', 'JJ'), ('human', 'JJ'), ('language', 'NN')]

Chunking:
 (S
  Natural/JJ
  Language/NNP
  Processing/NNP
  important/JJ
  field/NN
  (ORGANIZATION Artificial/NNP Intelligence/NNP)
  helps/VBZ
  machines/NNS
  understand/JJ
  human/JJ
  language/NN)
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
```

```
Dependency Parsing:
Natural -> compound -> Language
Language -> compound -> Processing
Processing -> nsubj -> is
is -> ROOT -> is
an -> det -> field
important -> amod -> field
field -> attr -> is
of -> prep -> field
Artificial -> compound -> Intelligence
Intelligence -> pobj -> of
. -> punct -> is

 -> dep -> .
It -> nsubj -> helps
helps -> ROOT -> helps
machines -> nsubj -> understand
understand -> ccomp -> helps
human -> amod -> language
language -> dobj -> understand
. -> punct -> helps
```

Observation:

- Preprocessing significantly reduces noise in text
- Stopword removal improves efficiency
- Stemming may produce non-meaningful roots (e.g., trouble → troubl)
- POS tagging helps understand sentence structure
- Dependency parsing gives deeper linguistic relationships

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:**	**Enrolment No:92410133020**

Post Lab Exercise:

1. Take your own document of 10 sentences and perform the same tasks. Attach code and screenshot of your output.

   text2 = """Machine learning is powerful. It is used in many applications.
   Data preprocessing is very important. NLP helps computers understand text.
   AI is the future. Students are learning AI. Python is popular.
   Models require clean data. Preprocessing improves accuracy.
   Stopwords are removed. Stemming reduces words."""

```
Chunking2:
 (S
  (GPE Machine/NN)
  learning/VBG
  powerful/JJ
  used/VBN
  many/JJ
  applications/NNS
  (PERSON Data/NNP)
  preprocessing/NN
  important/JJ
  (ORGANIZATION NLP/NNP)
  helps/VBZ
  computers/NNS
  understand/VBP
  text/JJ
  AI/NNP
  future/NN
  Students/NNS
  learning/VBG
  (ORGANIZATION AI/NNP Python/NNP)
  popular/JJ
  Models/NNP
  require/VBP
  clean/JJ
  data/NNS
  Preprocessing/VBG
  improves/NNS
  accuracy/NN
  (PERSON Stopwords/NNP)
  removed/VBD
  Stemming/NNP
  reduces/NNS
  words/NNS)
```

```
Dependency Parsing2:
Machine -> compound -> learning
learning -> nsubj -> is
is -> ROOT -> is
powerful -> acomp -> is
. -> punct -> is
It -> nsubjpass -> used
is -> auxpass -> used
used -> ROOT -> used
in -> prep -> used
many -> amod -> applications
applications -> pobj -> in
. -> punct -> used

 -> dep -> .
Data -> compound -> preprocessing
preprocessing -> nsubj -> is
is -> ROOT -> is
very -> advmod -> important
important -> acomp -> is
. -> punct -> is
NLP -> nsubj -> helps
helps -> ROOT -> helps
computers -> nsubj -> understand
understand -> ccomp -> helps
text -> dobj -> understand
. -> punct -> helps
```

```
 -> dep -> .
AI -> nsubj -> is
is -> ROOT -> is
the -> det -> future
future -> attr -> is
. -> punct -> is
Students -> nsubj -> learning
are -> aux -> learning
learning -> ROOT -> learning
AI -> dobj -> learning
. -> punct -> learning
Python -> nsubj -> is
is -> ROOT -> is
popular -> acomp -> is
. -> punct -> is

 -> dep -> .
Models -> nsubj -> require
require -> ROOT -> require
clean -> amod -> data
data -> dobj -> require
. -> punct -> require
Preprocessing -> nsubj -> improves
improves -> ROOT -> improves
accuracy -> dobj -> improves
. -> punct -> improves

 -> dep -> .
Stopwords -> nsubjpass -> removed
are -> auxpass -> removed
removed -> ROOT -> removed
. -> punct -> removed
Stemming -> nsubj -> reduces
reduces -> ROOT -> reduces
words -> dobj -> reduces
. -> punct -> reduces
```

2. Write your observation for stemming and lemmatization you obtained for the sentences.

Feature	Stemming	Lemmatization
Method	Rule-based cutting	Dictionary-based
Output	May not be real word	Real meaningful word
Example	troubling → troubl	troubling → trouble